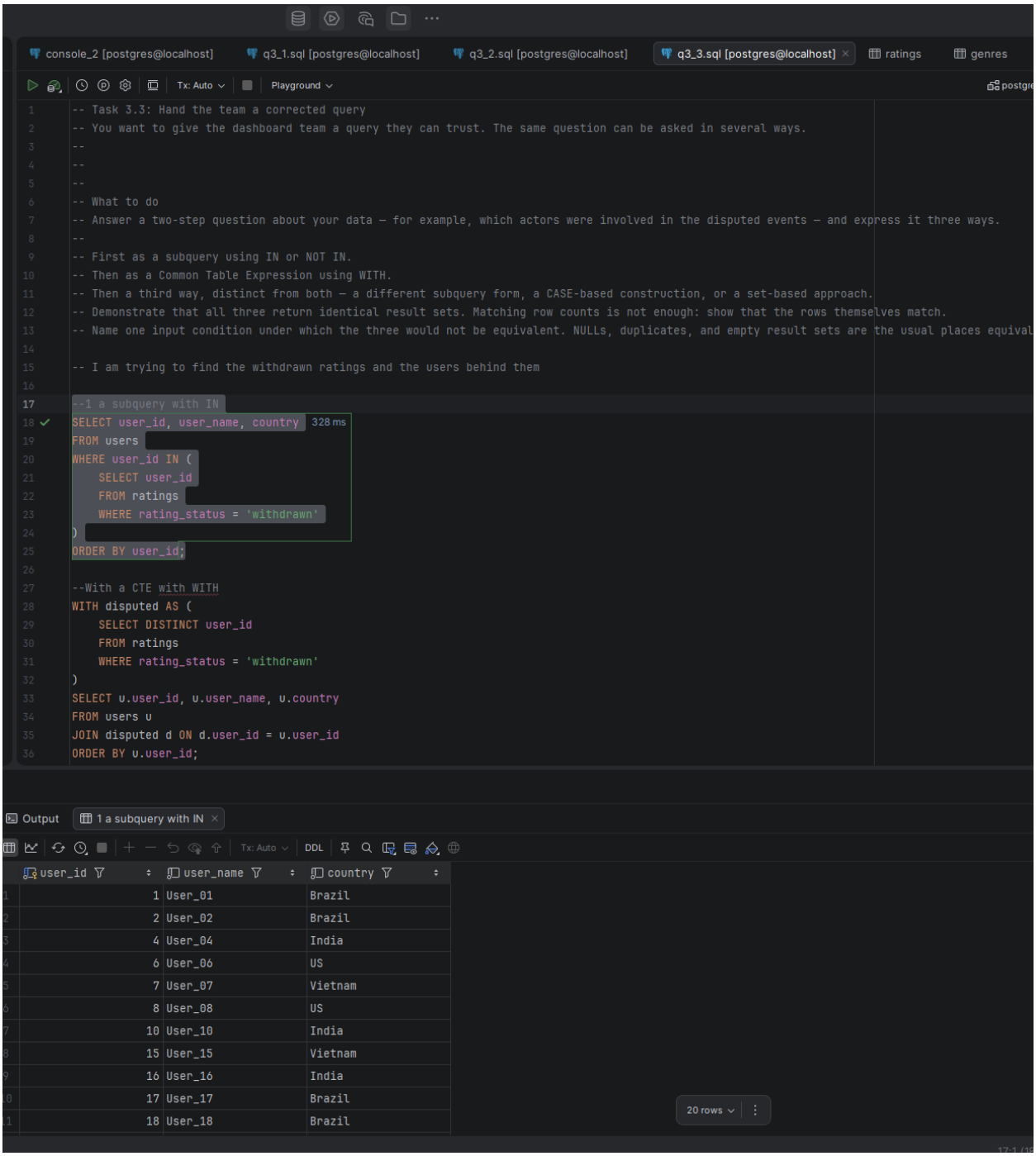
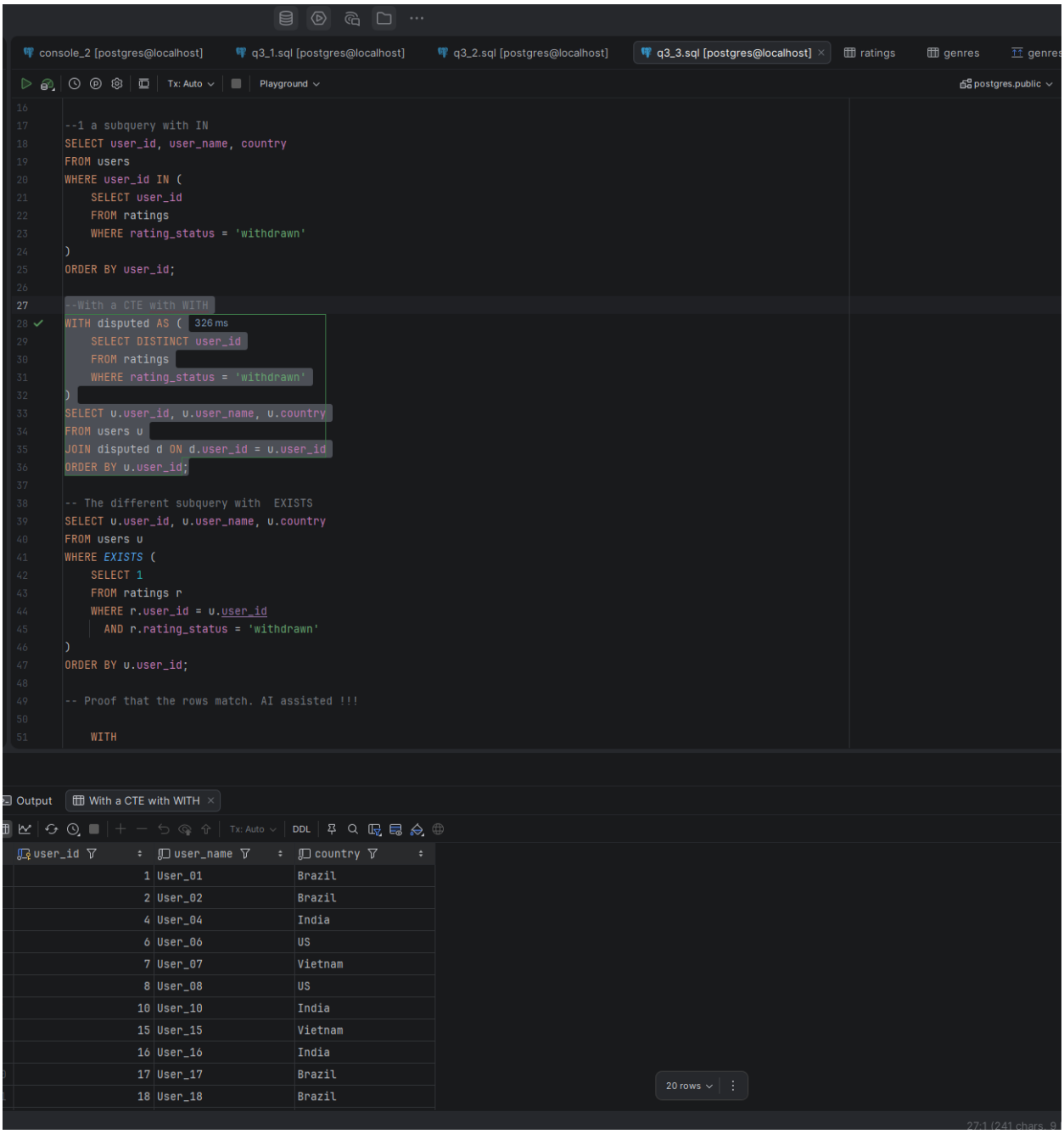
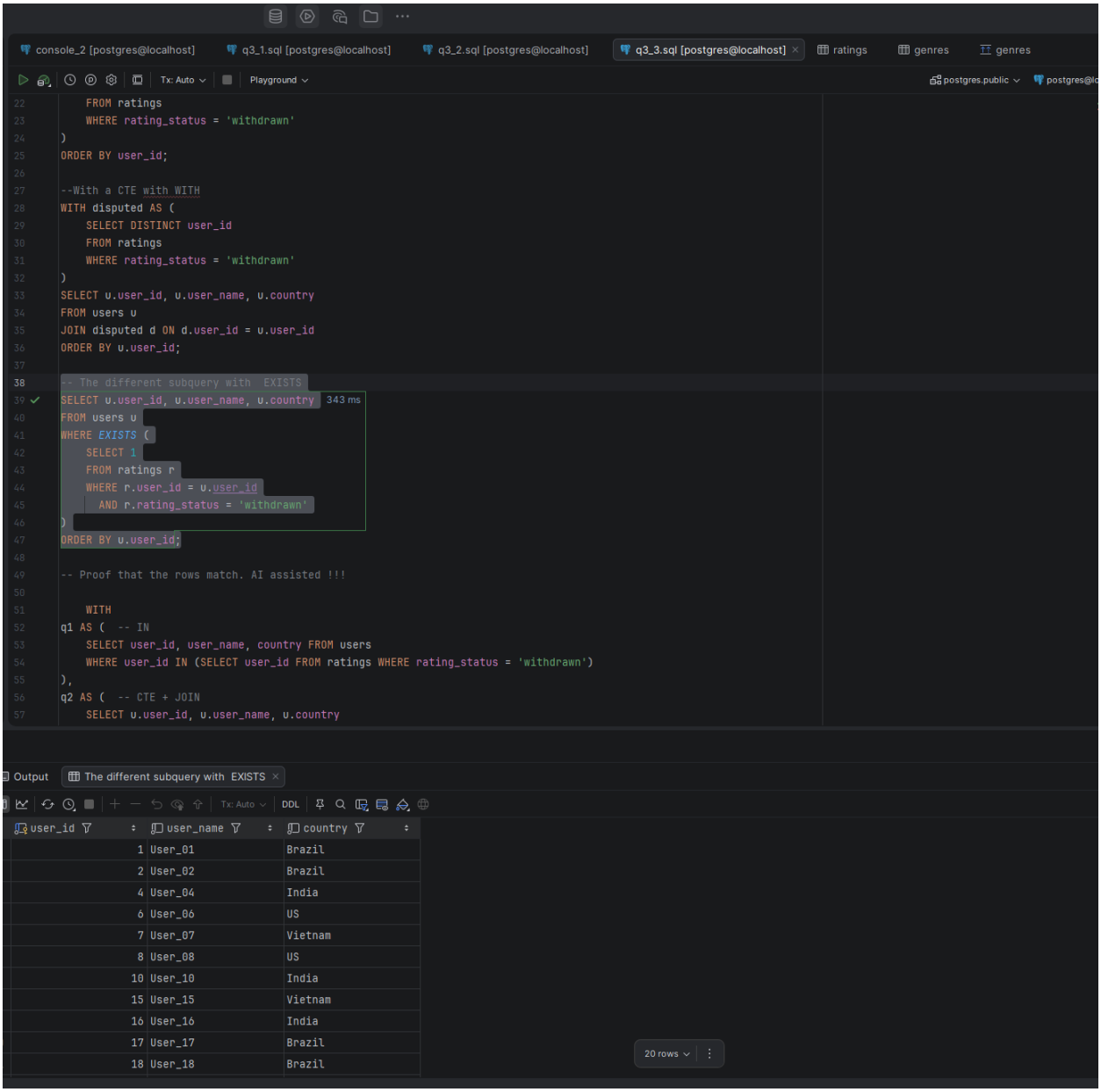
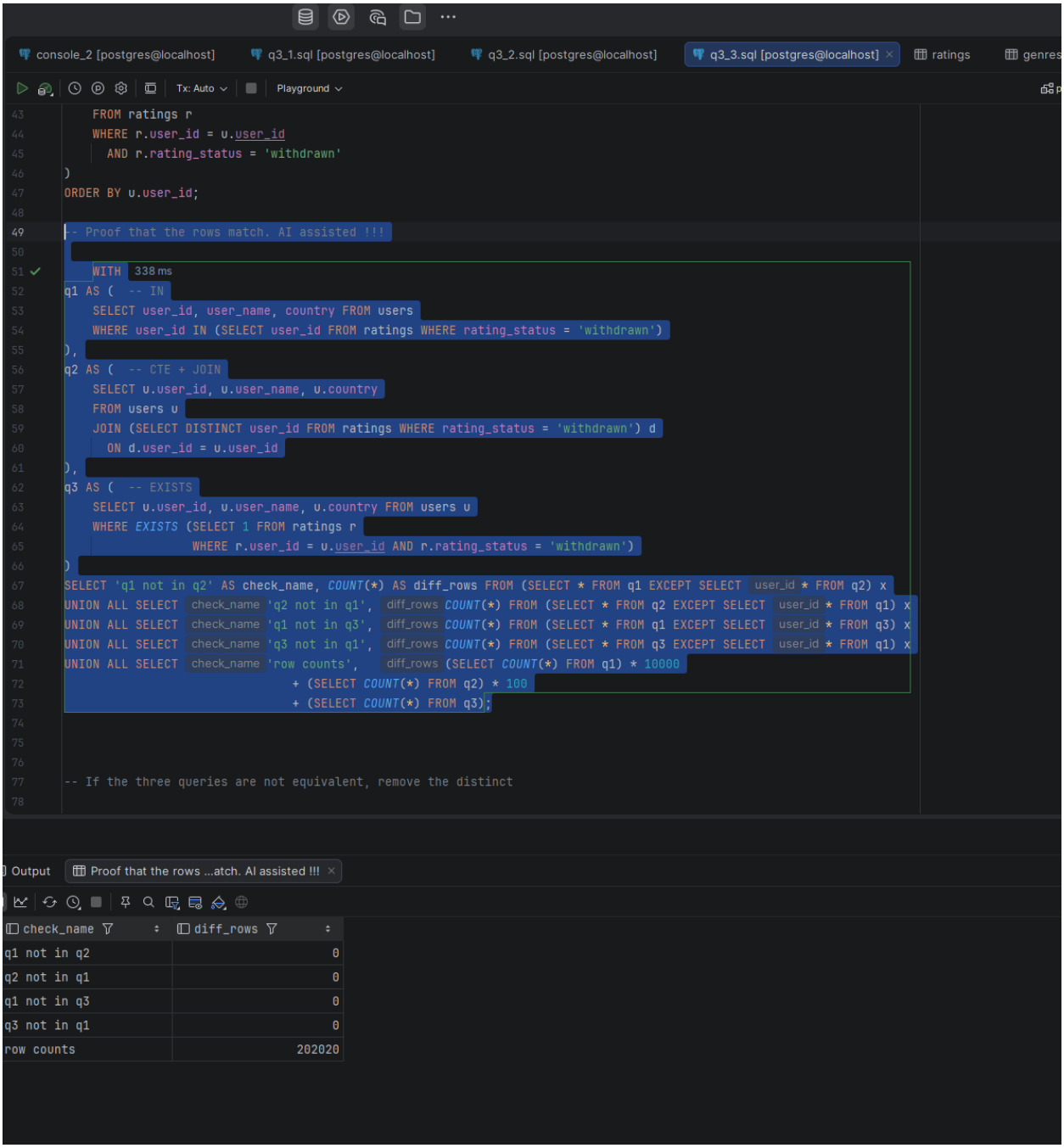
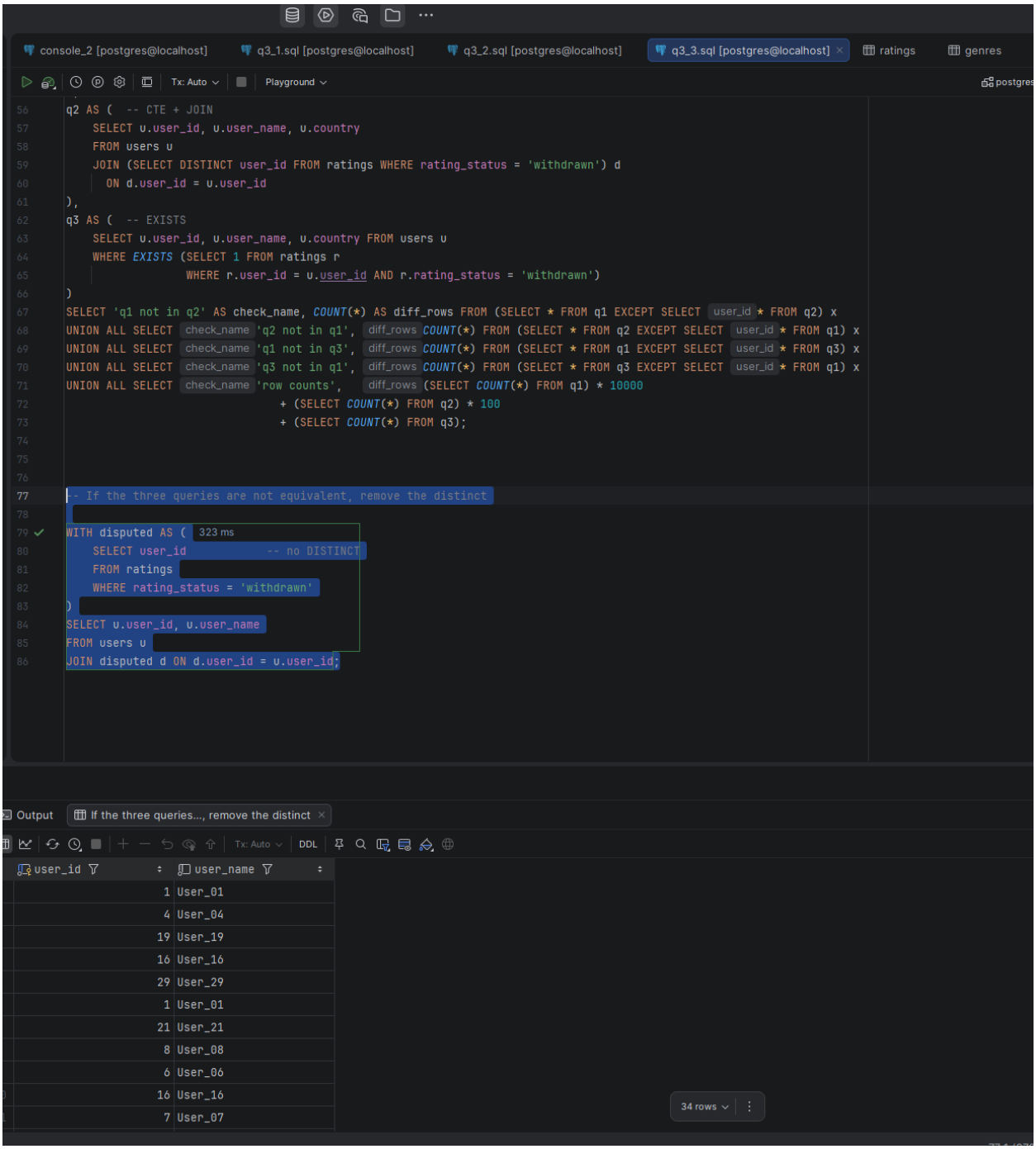

console_2 [postgres@localhost]
q3_1.sql [postgres@localhost]
q3_2.sql [postgres@localhost]
q3_3.sql [postgres@localhost]
ratings
genres
Tx: Auto
Playground
-- Task 3.3: Hand the team a corrected query
-- You want to give the dashboard team a query they can trust. The same question can be asked in several ways.
--
--
--
-- What to do
-- Answer a two-step question about your data — for example, which actors were involved in the disputed events — and express it three ways.
--
-- First as a subquery using IN or NOT IN.
-- Then as a Common Table Expression using WITH.
-- Then a third way, distinct from both — a different subquery form, a CASE-based construction, or a set-based approach.
-- Demonstrate that all three return identical result sets. Matching row counts is not enough: show that the rows themselves match.
-- Name one input condition under which the three would not be equivalent. NULLs, duplicates, and empty result sets are the usual places equival
-- I am trying to find the withdrawn ratings and the users behind them
--1 a subquery with IN
328 ms
SELECT user_id, user_name, country
FROM users
WHERE user_id IN (
SELECT user_id
FROM ratings
WHERE rating_status = 'withdrawn'
)
ORDER BY user_id;
--With a CTE with WITH
WITH disputed AS (
SELECT DISTINCT user_id
FROM ratings
WHERE rating_status = 'withdrawn'
)
SELECT u.user_id, u.user_name, u.country
FROM users u
JOIN disputed d ON d.user_id = u.user_id
ORDER BY u.user_id;
Output
1 a subquery with IN
Tx: Auto
DDL
user_id
user_name
country
1 User_01 Brazil
2 User_02 Brazil
4 User_04 India
6 User_06 US
7 User_07 Vietnam
8 User_08 US
10 User_10 India
15 User_15 Vietnam
16 User_16 India
17 User_17 Brazil
18 User_18 Brazil
20 rows


console_2 [postgres@localhost]
q3_1.sql [postgres@localhost]
q3_2.sql [postgres@localhost]
q3_3.sql [postgres@localhost]
ratings
genres
Tx: Auto
Playground
postgres.public
--1 a subquery with IN
SELECT user_id, user_name, country
FROM users
WHERE user_id IN (
SELECT user_id
FROM ratings
WHERE rating_status = 'withdrawn'
)
ORDER BY user_id;
--With a CTE with WITH
WITH disputed AS (326 ms
SELECT DISTINCT user_id
FROM ratings
WHERE rating_status = 'withdrawn'
)
SELECT u.user_id, u.user_name, u.country
FROM users u
JOIN disputed d ON d.user_id = u.user_id
ORDER BY u.user_id;
-- The different subquery with EXISTS
SELECT u.user_id, u.user_name, u.country
FROM users u
WHERE EXISTS (
SELECT 1
FROM ratings r
WHERE r.user_id = u.user_id
AND r.rating_status = 'withdrawn'
)
ORDER BY u.user_id;
-- Proof that the rows match. AI assisted !!!
WITH
Output
With a CTE with WITH
DDL
user_id
user_name
country
1 User_01 Brazil
2 User_02 Brazil
4 User_04 India
6 User_06 US
7 User_07 Vietnam
8 User_08 US
10 User_10 India
15 User_15 Vietnam
16 User_16 India
17 User_17 Brazil
18 User_18 Brazil
20 rows


console_2 [postgres@localhost]
q3_1.sql [postgres@localhost]
q3_2.sql [postgres@localhost]
q3_3.sql [postgres@localhost]
ratings
genres
genres
Tx: Auto
Playground
postgres.public
FROM ratings
WHERE rating_status = 'withdrawn'
)
ORDER BY user_id;
--With a CTE with WITH
WITH disputed AS (
SELECT DISTINCT user_id
FROM ratings
WHERE rating_status = 'withdrawn'
)
SELECT u.user_id, u.user_name, u.country
FROM users u
JOIN disputed d ON d.user_id = u.user_id
ORDER BY u.user_id;
-- The different subquery with EXISTS
SELECT u.user_id, u.user_name, u.country
343 ms
FROM users u
WHERE EXISTS (
SELECT 1
FROM ratings r
WHERE r.user_id = u.user_id
AND r.rating_status = 'withdrawn'
)
ORDER BY u.user_id;
-- Proof that the rows match. AI assisted !!!
WITH
q1 AS (-- IN
SELECT user_id, user_name, country FROM users
WHERE user_id IN (SELECT user_id FROM ratings WHERE rating_status = 'withdrawn')
),
q2 AS (-- CTE + JOIN
SELECT u.user_id, u.user_name, u.country
Output
The different subquery with EXISTS
Tx: Auto
DDL
user_id
user_name
country
1 User_01 Brazil
2 User_02 Brazil
4 User_04 India
6 User_06 US
7 User_07 Vietnam
8 User_08 US
10 User_10 India
15 User_15 Vietnam
16 User_16 India
17 User_17 Brazil
18 User_18 Brazil
20 rows


console_2 [postgres@localhost]
q3_1.sql [postgres@localhost]
q3_2.sql [postgres@localhost]
q3_3.sql [postgres@localhost]
ratings
genres
Tx: Auto
Playground
FROM ratings r
WHERE r.user_id = u.user_id
AND r.rating_status = 'withdrawn'
)
ORDER BY u.user_id;
-- Proof that the rows match. AI assisted !!!
WITH 338 ms
q1 AS (-- IN
SELECT user_id, user_name, country FROM users
WHERE user_id IN (SELECT user_id FROM ratings WHERE rating_status = 'withdrawn')
),
q2 AS (-- CTE + JOIN
SELECT u.user_id, u.user_name, u.country
FROM users u
JOIN (SELECT DISTINCT user_id FROM ratings WHERE rating_status = 'withdrawn') d
ON d.user_id = u.user_id
),
q3 AS (-- EXISTS
SELECT u.user_id, u.user_name, u.country FROM users u
WHERE EXISTS (SELECT 1 FROM ratings r
WHERE r.user_id = u.user_id AND r.rating_status = 'withdrawn')
)
SELECT 'q1 not in q2' AS check_name, COUNT(*) AS diff_rows FROM (SELECT * FROM q1 EXCEPT SELECT user_id * FROM q2) x
UNION ALL SELECT check_name 'q2 not in q1', diff_rows COUNT(*) FROM (SELECT * FROM q2 EXCEPT SELECT user_id * FROM q1) x
UNION ALL SELECT check_name 'q1 not in q3', diff_rows COUNT(*) FROM (SELECT * FROM q1 EXCEPT SELECT user_id * FROM q3) x
UNION ALL SELECT check_name 'q3 not in q1', diff_rows COUNT(*) FROM (SELECT * FROM q3 EXCEPT SELECT user_id * FROM q1) x
UNION ALL SELECT check_name 'row counts', diff_rows (SELECT COUNT(*) FROM q1) * 10000
+ (SELECT COUNT(*) FROM q2) * 100
+ (SELECT COUNT(*) FROM q3);
-- If the three queries are not equivalent, remove the distinct
Output
Proof that the rows ...atch. AI assisted !!!
check_name
diff_rows
q1 not in q2 0
q2 not in q1 0
q1 not in q3 0
q3 not in q1 0
row counts 202020


console_2 [postgres@localhost]
q3_1.sql [postgres@localhost]
q3_2.sql [postgres@localhost]
q3_3.sql [postgres@localhost]
ratings
genres
Tx: Auto
Playground
q2 AS (-- CTE + JOIN
SELECT u.user_id, u.user_name, u.country
FROM users u
JOIN (SELECT DISTINCT user_id FROM ratings WHERE rating_status = 'withdrawn') d
ON d.user_id = u.user_id
),
q3 AS (-- EXISTS
SELECT u.user_id, u.user_name, u.country FROM users u
WHERE EXISTS (SELECT 1 FROM ratings r
WHERE r.user_id = u.user_id AND r.rating_status = 'withdrawn')
)
SELECT 'q1 not in q2' AS check_name, COUNT(*) AS diff_rows FROM (SELECT * FROM q1 EXCEPT SELECT user_id * FROM q2) x
UNION ALL SELECT check_name 'q2 not in q1', diff_rows COUNT(*) FROM (SELECT * FROM q2 EXCEPT SELECT user_id * FROM q1) x
UNION ALL SELECT check_name 'q1 not in q3', diff_rows COUNT(*) FROM (SELECT * FROM q1 EXCEPT SELECT user_id * FROM q3) x
UNION ALL SELECT check_name 'q3 not in q1', diff_rows COUNT(*) FROM (SELECT * FROM q3 EXCEPT SELECT user_id * FROM q1) x
UNION ALL SELECT check_name 'row counts', diff_rows (SELECT COUNT(*) FROM q1) * 10000
+ (SELECT COUNT(*) FROM q2) * 100
+ (SELECT COUNT(*) FROM q3);
-- If the three queries are not equivalent, remove the distinct
WITH disputed AS (323 ms
SELECT user_id -- no DISTINCT
FROM ratings
WHERE rating_status = 'withdrawn'
)
SELECT u.user_id, u.user_name
FROM users u
JOIN disputed d ON d.user_id = u.user_id;
Output
If the three queries..., remove the distinct
Tx: Auto
DDL
user_id
user_name
1 User_01
4 User_04
19 User_19
16 User_16
29 User_29
1 User_01
21 User_21
8 User_08
6 User_06
16 User_16
7 User_07
34 rows